

Unveiling LLM Hallucination in Recommending Third-party Packages

ANONYMOUS AUTHOR(S)

In this paper, ...

1 Introduction

1. LLM widely used for developers
2. LLM hallucination
3. Hallucinated packages can cause supply chain attack.
4. How hallucinated package is not yet fully explored
5. The purpose of this study is to.
6. Specifically, we design xxx.
7. The results reveal that
8. Our contribution are as follows.

2 Background

2.1 Hallucination

2.2 Package Squatting

3 Empirical Study Design

3.1 Research Questions

3.2 Dataset Setup

For the investigation of RQ1 and RQ2, we constructed a functional question dataset designed to trigger package recommendation behaviors in large language models. Furthermore, we developed a semantically rewritten version of this dataset to mitigate the potential impact of data leakage.

3.2.1 Initial Dataset Collection. To conduct an in-depth analysis of the hallucination phenomenon that may occur when large language models recommend Python packages, this study first constructed a functional question dataset covering typical development task scenarios. The dataset was sourced from Stack Overflow, where we systematically identified questions related to major Python application domains that have the potential to trigger package recommendation behaviors, thereby establishing a reliable data foundation for the subsequent empirical study.

First, we adopted the co-occurrence-based tag filtering method proposed by Huang et al. [1] and utilized the publicly available Stack Overflow dataset provided by Google BigQuery [2] to extract all tags that co-occurred with the “python” tag more than 1,000 times, resulting in a total of 212 tags. On this basis, drawing on the topic classification strategy used by Du et al. [3] in constructing code generation benchmark tasks, as well as the categorization of common Python development topics summarized by Tahmoore et al. [4], we manually reviewed these 212 high-frequency co-occurring tags. Specifically, we assessed whether each tag belonged to one of the twelve mainstream Python development domains, whether it was likely to involve the use or recommendation of software packages, and whether it could potentially give rise to functional questions of the form “how to implement a specific functionality.” Through this filtering process, we ultimately retained 114 tags. Their detailed categorization is presented in Table 1.

After completing the tag filtering process, we extracted from the public dataset all questions that contained both the “python” tag and at least one of the filtered tags, resulting in a total of 287,944 questions. Subsequently, we randomly sampled 200 questions for manual analysis. The results indicated that typical functional questions related to software package recommendation behaviors often contain functional expression keywords such as “how to” or “how can I.” Based on this observation, we filtered the entire set of questions using these keywords, obtaining 49,967 candidate questions with potential functional requirements.

To further identify genuinely functional questions from the 49,967 candidate questions, we first conducted an in-depth analysis of a randomly selected subset of 200 questions and summarized the typical characteristics of functional questions. Based on these findings, we designed targeted classification prompts and invoked the large language model DeepSeek R1 to evaluate each candidate question individually. Through this process, we obtained a refined dataset consisting of 17,696 functional questions. The detailed design of the prompts is provided in Appendix A.

After completing the preliminary screening of functional questions, we further removed those without an accepted answer to ensure that each retained question was associated with a community-validated solution. Subsequently, we extracted the accepted answer for each question from the public dataset and matched it with the corresponding question, thereby constructing a question–answer pair dataset for subsequent analysis.

To determine whether each accepted answer involved specific third-party Python libraries, we designed corresponding prompts (the details of which are provided in Appendix B) and then employed the large language model DeepSeek R1 to analyze all accepted answers, automatically extracting the names of third-party libraries mentioned in the responses (i.e., software packages that can be installed from PyPI via pip). For question–answer pairs where the model returned an empty result—indicating that the answer did not involve any third-party libraries—we excluded them from subsequent processing. After this filtering step, we ultimately obtained 4,382 functional question–answer pairs containing information on third-party library usage, which constitute the final functional question dataset used in this study.

3.2.2 Semantic Rewriting Dataset Construction. Since the above question–answer data were all sourced from the publicly available Stack Overflow community, some of the content may have been encountered by large language models during their training process. Therefore, directly conducting evaluations based on the original questions may underestimate the actual occurrence rate of hallucinated packages. To obtain more reliable experimental results, we performed semantic rewriting on all questions to mitigate the potential impact of data leakage.

Specifically, we invoked the large language model DeepSeek R1 to generate five semantically equivalent rewritten versions for each question. To achieve a balance between semantic preservation and variation in expression—avoiding rewrites that were either overly similar to the original question or that deviated excessively in meaning—we further introduced a filtering strategy based on sentence embedding similarity. For each original question and its five corresponding rewrites, we employed the Sentence-BERT model all-mpnet-base-v2 to generate sentence embeddings and computed the cosine similarity between the embedding of the original question and that of each rewritten version, resulting in five similarity scores. We then ranked the five rewritten versions in descending order of similarity and selected the version whose similarity score was at the median position as the final semantic rewrite for that question. Ultimately, we constructed a semantically rewritten question dataset that

corresponds one-to-one with the original functional question dataset, which was used for subsequent evaluation of hallucinated packages under controlled conditions to mitigate potential data leakage effects.

In summary, this study constructed a functional question–answer dataset covering multiple typical Python development scenarios based on publicly available Stack Overflow data, along with a corresponding semantically rewritten question dataset.

3.3 RQ Setup

3.3.1 RQ1 Setup. To evaluate the frequency with which large language models generate hallucinated packages when recommending Python libraries, we designed a unified and comprehensive experimental procedure. We selected several mainstream models, including ChatGPT-4, DeepSeek-R1, Qwen3, and Claude-Opus-4. The titles of the questions from the functional question dataset constructed in the previous stage were used as input prompts, and each model was asked to recommend one or more third-party Python libraries that could be used to address the given question. To distinguish cases in which a model determines that no third-party library is required, we specified in the prompt that if the model judges a question does not require any third-party libraries, it should return a special token, `no_package`.

During the experiment, we first designed a unified prompt for all models, the full content of which is provided in Appendix C. When querying the models, we observed that some of them occasionally returned package names containing characters not permitted by PyPI (e.g., parentheses, square brackets, equal signs, or spaces). To ensure the validity and consistency of the experimental results, we implemented an automated script to examine the model outputs and automatically reissue the query whenever illegal characters were detected, repeating this process until the returned package names contained no disallowed characters.

In addition, for some package names generated by the models that contained dot characters (e.g., `scipy.sparse`), we found that such submodules cannot be directly installed as top-level packages. To address this issue, we designed a more restrictive prompt (see Appendix D), explicitly requiring the models to return only top-level package names—for example, replacing `scipy.sparse` with `scipy`. Whenever a returned result contained a dot, we reissued the query using this stricter prompt. However, we further observed that even after multiple rounds of querying, a small number of models occasionally still returned package names containing dots (e.g., `*RPi.GPIO*`). In such cases, we chose to retain these results and handle them uniformly in the subsequent stages of the workflow.

After completing the queries to all models, we performed a unified post-processing procedure on the returned package names. First, we removed all records whose returned value was `no_package`, retaining only those questions for which the models explicitly provided recommended package names. Subsequently, we queried each recommended package individually using the official PyPI JSON API (https://pypi.org/pypi/{pkg_name}/json) to verify whether it actually exists. When the request returned a status code of 200, we regarded the package as a real package. During this process, we observed that the previously retained package names containing dot characters also returned a status code of 200 from the API, indicating that these dotted package names indeed exist on PyPI. Therefore, we treated them as real packages in our analysis as well.

Finally, we conducted an additional screening of the package names that failed the PyPI verification and were initially identified as hallucinated packages. Although the prompts explicitly required the models to recommend only third-party libraries, some models occasionally returned Python standard libraries (e.g., `*os*`, `*sys*`) for a small number of questions. Since these standard libraries are not hosted on PyPI, they would be misclassified

as hallucinated packages in the previous verification step. To eliminate such cases, we utilized the `importlib.util.find_spec()` interface to determine whether a given name belongs to the Python standard library and removed all identified standard library entries accordingly. After this filtering step, we ultimately obtained a result set containing only hallucinated packages.

3.3.2 RQ2 Setup. To further investigate whether hallucinated packages generated by large language models pose a risk of being downloaded by real developers, we selected all 91 hallucinated package names recommended by ChatGPT-4 and attempted to register and upload them to the PyPI platform. For each package, we created standardized `setup.py`, `__init__.py`, and `README.md` files. The file contents explicitly stated that the package was intended solely for academic research purposes, contained no functional code, and performed no malicious operations. The `README` file provided detailed information regarding the research background, research participants, contact information, and research objectives to ensure that the experimental process complied with academic ethical standards. All packages were uploaded manually by the authors.

We began uploading these hallucinated packages on May 18, 2025, and completed the entire upload process on May 22, 2025. Throughout this period, a total of nine PyPI accounts were used to register and upload the packages in batches. Among the 91 packages, 55 were successfully registered, while the remaining 36 failed due to various reasons. The causes of failure included certain package names being protected by PyPI, names being considered too similar to existing projects, and account restrictions imposed by the platform due to bulk upload activities being flagged as suspicious behavior.

On June 10, 2025, we received an email from the PyPI administrators requesting the immediate removal of all packages related to the experiment. Upon receiving the notification, we completed the removal of all packages between June 12 and June 13.

After uploading the hallucinated packages to PyPI, we continuously monitored their subsequent download activities to assess their real-world impact. We collected download statistics for each package using two third-party data analytics platforms, `pepy.tech` and `pypistats.org`, which provide daily download counts as well as cumulative downloads over different time ranges. The corresponding results and analyses will be presented in detail in the Results section.

3.3.3 RQ3 Setup.

4 Results

4.1 RQ1

1. Fig: x-axis is models, y-axis are hallucinated package absolute number and relative hallucination percentage.
2. Fig: x-axis is hallucinated packages ranked by their hallucination times among models, y-axis is the hallucination number.

4.2 RQ2

1. Package name characteristics.
 - 1.1 Syntactic analysis
 - 1.2 Package domain analysis
2. Analysis on questions that cause hallucinated package name.

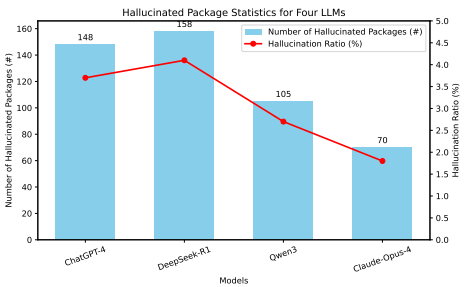


Fig. 1. The number and ratio of hallucinated packages generated by four LLMs.

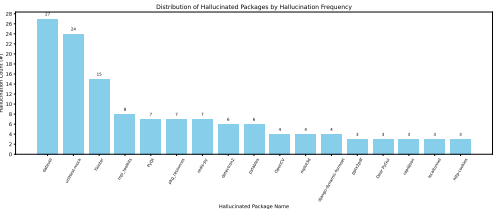


Fig. 2. Distribution of hallucinated packages by hallucination frequency.

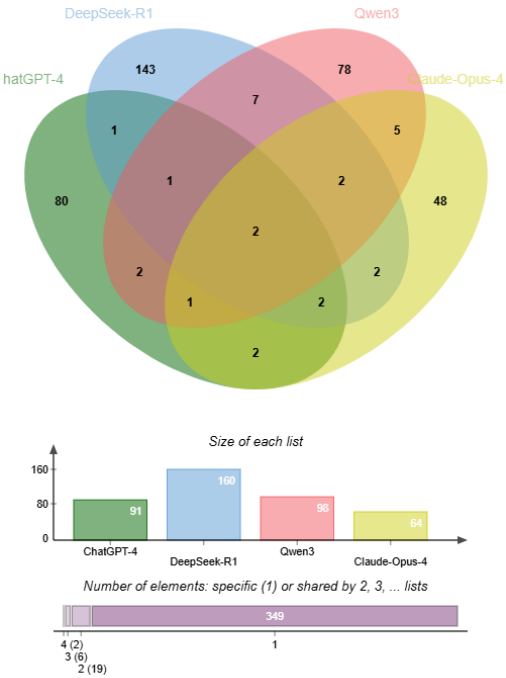


Fig. 3. Venn diagram of hallucinated package overlaps among LLMs.

4.3 RQ3

x-axis is time, y-axis is total download times, and average download number.

TODO fig. download numbers with regard to package names

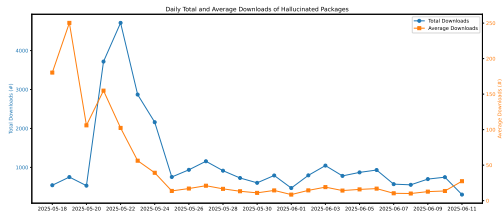


Fig. 4. Daily total and average downloads of hallucinated packages.

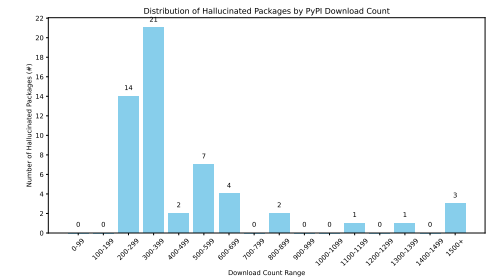


Fig. 5. Distribution of hallucinated packages by total PyPI downloads.

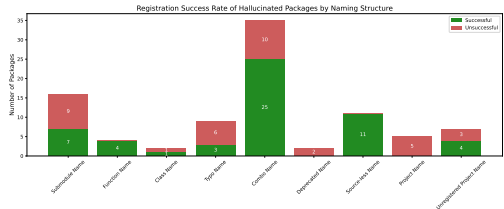


Fig. 6. Registration success rate of hallucinated packages across naming structures.

Table 1. PyPI Package Names

Feature Category	Example Package Names	Count
Submodule Name	mpl_toolkits, mplot3d, gridsf, pkg_resources, sklearn-base, django-forms, MySQLdb, yaml, dateutil	9
Function/Usage Name	odeint, django-contenttypes	2
Class Name		
Typo Name	urllib3, shutilwich, maze-solvers	3
Combo Name	django-classy-views, flask-autocomple	20+
Deprecated Name	pytest-capturelog	1
Source-less Name	translate-api, pycapsule-api, manifestin	3
Project Name	redis-py, pyqt, opencv	3
Unregistered Project Name	zeromq, OpenSSL, package_control, h2oai, tastypie, sslsplit	6
total		XXX

Table 2. Domain Type

Domain Type	Example Package Names	Count
Video		
Web		

4.4 RQ4

Put Table, illustrate characteristics.

Table 3. Register Success Rate of Hallucinated Package

Feature Category	Count (%)
Submodule Name	24 (4%)

Table 4. AUROC (%) of different methods under varying member-to-non-member ratios.

Ratio	Method	Pythia 2.8B	GPT-Neo 2.7B	StableLM-Alpha 3B	GPT-J 6B
1:1	LOSS	38.4	43.7	33.3	43.3
	ZLIB	33.2	35.5	32.0	35.7
	MIN-K	38.8	41.8	34.2	41.1
	DC-PDD	50.1	41.2	43.7	41.3
	TOOL	61.3	59.7	61.3	59.7
1:5	LOSS	40.3	45.3	33.9	44.8
	ZLIB	34.0	36.2	32.5	36.3
	MIN-K	40.2	43.3	34.9	42.4
	DC-PDD	51.8	44.0	44.6	43.2
	TOOL	61.2	59.4	61.1	61.5
5:1	LOSS	40.0	43.8	34.9	43.6
	ZLIB	33.1	35.0	31.9	35.2
	MIN-K	39.9	42.2	35.1	41.6
	DC-PDD	49.4	40.9	44.1	39.6
	TOOL	62.0	60.7	62.0	63.1

5 Mitigation

5.1 Mitigation Methodology

- 5.1.1 *Enhanced Prompting.*
- 5.1.2 *Machine-learning-based Detection.*
- 5.1.3 *Context-Augmented-Prompt Validation.* Typo squatted package?

5.2 Evaluation Setup

- **Mitigation Effectiveness (RQ5):** How do mitigation techniques address the hallucinated package problem?

5.3 RQ5

5.4 Reservation of Squating/Hallucination Packages

6 Related Work

7 Conclusion

References